

Hashtable Collection Class in C# with Examples

In this article, I am going to discuss **Non-Generic Hashtable Collection Class in C#** with Examples. Please read our previous article where we discussed the [ArrayList Collection Class in C#](#) with Examples. At the end of this article, you will understand the following pointers.

1. [What are the Problems with Array and ArrayList Collection in C#?](#)
2. [What is a Hashtable in C# and How does it work?](#)
3. [Differences between Hashtable and ArrayList in C#?](#)
4. [How to Create a Non-Generic Hashtable Collection in C#?](#)
5. [How to Add Elements into a Hashtable Collection in C#?](#)
6. [How to Access a Non-Generic Hashtable Collection in C#?](#)
7. [How to Check the Availability of a key/value Pair in a Hashtable in C#?](#)
8. [How to Remove Elements from a Non-Generic Hashtable Collection in C#?](#)
9. [How to Assign and Update Values to a Hashtable with Indexer in C#?](#)
10. [How to Clone a Non-Generic Hashtable Collection in C#?](#)
11. [How to Copy a Hashtable to an Existing Array in C#?](#)

Before understanding the Hashtable in C#, let us first understand the problems that we face with Array and ArrayList collection in C# and how we overcome such problems with Hashtable.

What are the Problems with Array and ArrayList Collection in C#?

In the case of [Array](#) and [ArrayList](#) in C#, we access the elements from the collection using the index position. The index position of the elements starts from zero (0) to the number of elements – 1. But, it is very difficult for us to remember the index position of the element in order to access the values.

For example, let us say we have an employee array that contains the name, address, mobile, dept no, email id, employee id, salary, location, etc. Now if I want to know the email id or dept number of the employee then it is very difficult for me to use the index position. The following example shows this. Here we create a collection using ArrayList and then we are accessing the Location and EmailId by using the index position. Here, we need to remember the index position of each value which is a difficult task and without knowing the index position we cannot access the value.

```
using System;
using System.Collections;
namespace HasntableExample
{
    class Program
    {
        static void Main(string[] args)
        {
            ArrayList al = new ArrayList();
            al.Add(1001); //EId - Index Position = 0
            al.Add("James"); //Name - Index Position = 1
            al.Add("Manager"); //Job - Index Position = 2
            al.Add(3500); //Salary - Index Position = 3
            al.Add("Mumbai"); //Location - Index Position = 4
            al.Add("IT"); //Dept - Index Position = 5
        }
    }
}
```

```

        al.Add("a@a.com"); // Emailid - Index Position = 6
        //I want to print the Location, Index position is 4
        Console.WriteLine("Location : " + al[4]);
        //I want to print the Email ID, Index position is 6
        Console.WriteLine("Email ID : " + al[6]);
        Console.ReadKey();
    }
}
}

```

If you have a huge number of elements in the collection, then it is very difficult to remember the index position of each element. So, it would not be nice, instead of using the index position of the element, we can access the elements by using a key. This is where **Hashtable in C#** comes into the picture.

What is a Hashtable in C#?

The Hashtable in C# is a Non-Generic Collection that stores the element in the form of “Key-Value Pairs”. The data in the Hashtable are organized based on the hash code of the key. The key in the Hashtable is defined by us and more importantly, that key can be of any data type. Once we created the Hashtable collection, then we can access the elements by using the keys. The Hashtable class comes under the System.Collections namespace.

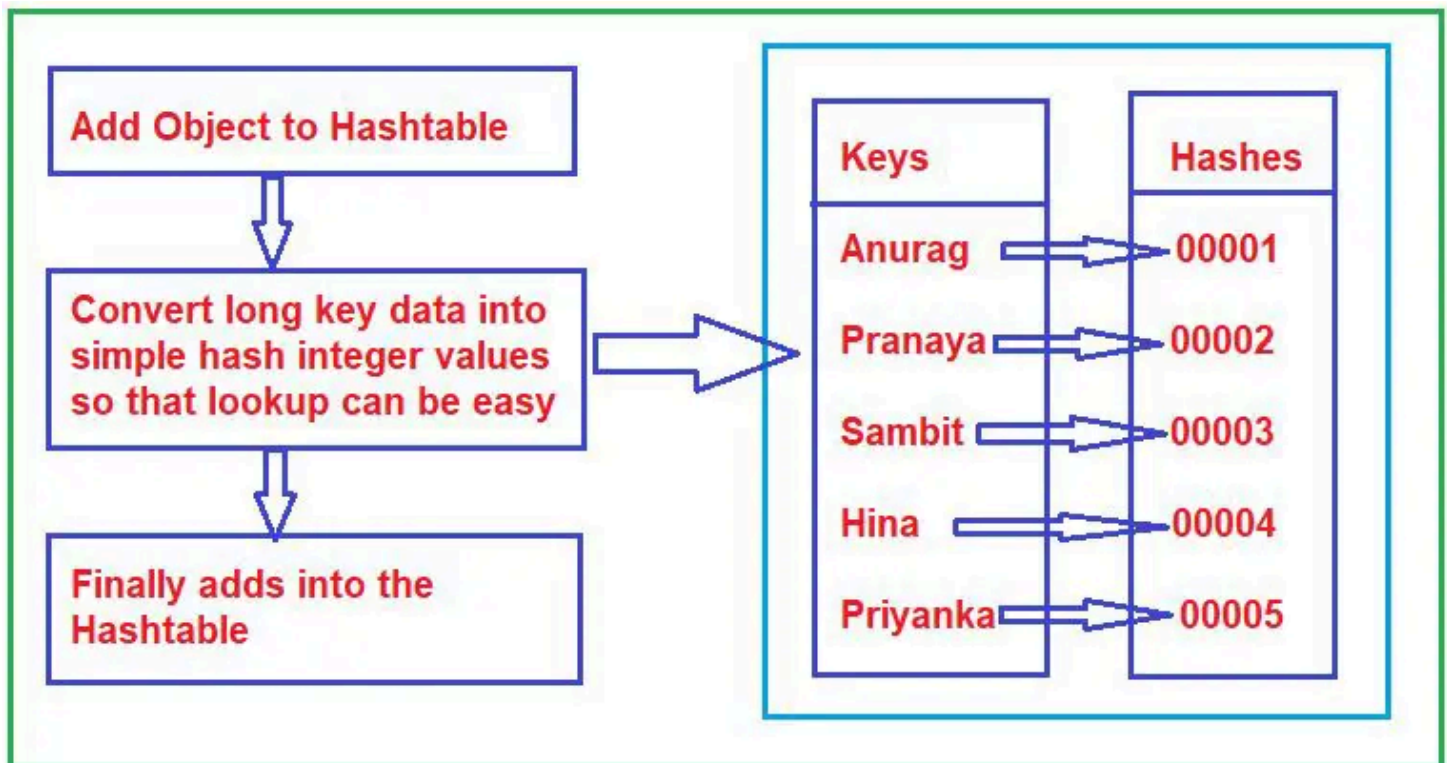
The Hashtable computes a hash code for each key. Then it uses that hash code to look up the elements very quickly which increases the performance of the application.

Hashtable Characteristics in C#

1. The Hashtable Collection Class in C# stores the elements in the form of key-value pairs.
2. Hashtable Class belongs to System.Collection namespace i.e. it is a Non-Generic Collection class.
3. It implements the IDictionary interface.
4. The Keys can be of any data type but they must be unique and not null.
5. The Hashtable accepts both null and duplicate values.
6. We can access the values by using the associated key.
7. The capacity of a Hashtable is the number of elements that a Hashtable can hold.
8. A hashtable is a non-generic collection, so we can store elements of the same type as well as of different types.

How Actually the Hashtable works in C#?

When we add elements to a hashtable like string, int, or complex types, then it converts the key data which can be a string, integer, numeric, or anything in the world into simple hash integer values so that lookup can be easy. Once the conversion is done, then the data will be added to the hashtable collection. For a better understanding, please have a look at the below image.



One more point that you need to remember is that the performance of the hashtable is less as compared to the ArrayList because of this key conversion (converting the key to an integer hashcode).

Differences between ArrayList and Hashtable in C#:

1. **Lookup:** ArrayList can be only looked up via the index number which is generated internally. Hashtable can be looked up by a custom-defined key.
2. **Performance:** ArrayList is faster than hashtable because of extra tasks performed in hashtables i.e. hashing.
3. **Scenario:** If you want a key lookup use hashtable. If you just want to add and browser through a collection then use ArrayList.

How to Create a Non-Generic Hashtable Collection in C#?

The Non-Generic Hashtable class in C# provides 16 different types of constructors that we can use to create a hashtable as shown in the below image. You can use any of the overloaded constructors to create an instance of Hashtable.

```

public class Hashtable : ICollection, IEnumerable, IDictionary, ICloneable, IDeserializationCallback, ISerializable
{
    ...public Hashtable();
    ...public Hashtable(IDictionary d);
    ...public Hashtable(int capacity);
    ...public Hashtable(IEqualityComparer? equalityComparer);
    ...public Hashtable(int capacity, float loadFactor);
    ...public Hashtable(IHashCodeProvider? hcp, IComparer? comparer);
    ...public Hashtable(int capacity, IEqualityComparer? equalityComparer);
    ...public Hashtable(IDictionary d, float loadFactor);
    ...public Hashtable(IDictionary d, IEqualityComparer? equalityComparer);
    ...public Hashtable(IDictionary d, float loadFactor, IEqualityComparer? equalityComparer);
    ...public Hashtable(IDictionary d, IHashCodeProvider? hcp, IComparer? comparer);
    ...public Hashtable(int capacity, IHashCodeProvider? hcp, IComparer? comparer);
    ...public Hashtable(int capacity, float loadFactor, IEqualityComparer? equalityComparer);
    ...public Hashtable(int capacity, float loadFactor, IHashCodeProvider? hcp, IComparer? comparer);
    ...public Hashtable(IDictionary d, float loadFactor, IHashCodeProvider? hcp, IComparer? comparer);
    ...protected Hashtable(SerializationInfo info, StreamingContext context);
}

```

Here, we are going to use the overloaded version which does not take any parameter i.e. the following Hashtable constructor.

1. **public Hashtable():** It is used to initialize a new, empty instance of the Hashtable class using the default initial capacity, load factor, hash code provider, and comparer.

Now, let's see how to create a hashtable in C#:

Step1:

The Hashtable class belongs to System.Collections namespace. So, first, we need to include the System.Collections namespace in our program with the help of the "using" keyword as follows.

```
using System.Collections;
```

Step2:

Next, we need to create an instance of the Hashtable class using the Hashtable constructor as follows. Here, we are using the 0-argument constructor.

```
Hashtable hashtable = new Hashtable();
```

How to Add Elements into a Hashtable Collection in C#?

Now, if you want to add elements i.e. a key/value pair into the hashtable, then you need to use Add() method of the Hashtable class.

1. **Add(object key, object? value):** The Add(object key, object? value) method is used to add an element with the specified key and value into the Hashtable. Here, the parameter key specifies the key of the element to add and the parameter value specifies the value of the element to add. The value can be null.

For Example

```
hashtable.Add("EId", 1001);
hashtable.Add("Name", "James");
```

Even it is also possible to create a Hashtable using collection-initializer syntax as follows:

```
var cities = new Hashtable(){
    {"UK", "London, Manchester, Birmingham"},

```

```

        {"USA", "Chicago, New York, Washington"},
        {"India", "Mumbai, Delhi, BBSR"}
    };

```

How to access a Non-Generic Hashtable Collection in C#?

We can access the key/value pairs of the Hashtable in C# using two different ways. They are as follows:

Using Keys to Access Hashtable in C#:

You can access the individual value of the Hashtable in C# by using the keys. In this case, we need to pass the key as a parameter to find the respective value. If the specified key is not present, then the compiler will throw an exception. The syntax is given below.

```

hashtable["Eid"]
hashtable["Name"]

```

Using ForEach loop to Access Hashtable in C#:

We can also use a for-each loop to access the key/value pairs of a Hashtable collection in C# as follows.

```

foreach (object obj in hashtable.Keys)
{
    Console.WriteLine(obj + " : " + hashtable[obj]);
}

```

The elements in Hashtable are stored as DictionaryEntry objects. So, instead of an object, you can also use DictionaryEntry.

```

foreach (DictionaryEntry item in hashtable)
{
    Console.WriteLine($"Key: {item.Key}, Value: {item.Value}");
}

```

Example to Understand How to Create a Hashtable and Add Elements in C#:

For a better understanding of how to create a Hashtable and how to add elements to a Hashtable, and how to access the elements of the hashtable in C#, please have a look at the below example.

```

using System;
using System.Collections;
namespace HashtableExample
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating Hashtable collection with default constructor
            Hashtable hashtable = new Hashtable();
            //Adding elements to the Hash table using key value pair
            hashtable.Add("Eid", 1001); //Here key is Eid and value is 1001
            hashtable.Add("Name", "James"); //Here key is Name and value is James
            hashtable.Add("Salary", 3500); //Here key is Salary and value is 3500
        }
    }
}

```

Mumbai

a@a.com

```
hashtable.Add("Location", "Mumbai"); //Here key is Location and va
hashtable.Add("EmailID", "a@a.com"); //Here key is EmailID and val

//Printing the keys and their values using foreach loop
Console.WriteLine("Printing Hashtable using Foreach Loop");
foreach (object obj in hashtable.Keys)
{
    Console.WriteLine(obj + " : " + hashtable[obj]);
}
//Or
//foreach (DictionaryEntry de in hashtable)
//{
//    Console.WriteLine($"Key: {de.Key}, Value: {de.Value}");
//}
Console.WriteLine("\nPrinting Hashtable using Keys");
//I want to print the Location by using the Location key
Console.WriteLine("Location : " + hashtable["Location"]);
//I want to print the Email ID by using the EmailID key
Console.WriteLine("Emaild ID : " + hashtable["EmailID"]);
Console.ReadKey();
    }
}
```

Output:



```
Printing Hashtable using Foreach Loop
EmailID : a@a.com
EId : 1001
Name : James
Location : Mumbai
Salary : 3500

Printing Hashtable using Keys
Location : Mumbai
Emaild ID : a@a.com
```

Example to Add Elements to a Hashtable using Collection Initializer in C#:

In the below example, we are using Collection Initializer syntax instead of the Add method of the Hashtable collection class to add key-value pairs into the hashtable in C#.

```
using System;
using System.Collections;
namespace HasntableExample
```

```

{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating a Hashtable using collection-initializer syntax
            Hashtable citiesHashtable = new Hashtable(){
                {"UK", "London, Manchester, Birmingham"}, //Key:UK
                {"USA", "Chicago, New York, Washington"}, //Key:USA
                {"India", "Mumbai, New Delhi, Pune"} //Key:India
            };
            Console.WriteLine("Creating a Hashtable Using Collection-Initializer");
            foreach (DictionaryEntry city in citiesHashtable)
            {
                Console.WriteLine($"Key: {city.Key}, Value: {city.Value}");
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```

Creating a Hashtable Using Collection-Initializer
Key: India, Value: Mumbai, New Delhi, Pune
Key: USA, Value: Chicago, New York, Washington
Key: UK, Value: London, Manchester, Birmingham

```

How to Check the Availability of a key/value Pair in a Hashtable in C#?

If you want to check whether the key/value pair exists or not in the Hashtable, then you can use the following methods of the Hashtable class.

1. **Contains(object key):** The Contains(object key) method of the Hashtable is used to check whether the Hashtable contains a specific key. The parameter key to locating in the Hashtable object. It returns true if the Hashtable contains an element with the specified key; otherwise, false. If the key is null, then it will throw System.ArgumentNullException.
2. **ContainsKey(object key):** The ContainsKey(object key) method of the Hashtable is used to check if a given key is present in the Hashtable or not. The parameter key to locating in the Hashtable object. If the given key is present in the collection then it will return true else it will return false. If the key is null, then it will throw System.ArgumentNullException.
3. **ContainsValue(object value):** The ContainsValue(object value) Method of the Hashtable class is used to check if a value is present in the Hashtable or not. The parameter value to locate in the hashtable object. If the given value is present in the collection then it will return true else it will return false.

Let us understand this with an example. The following example shows how to use the Contains, ContainsKey, and ContainsValue methods of the Non-Generic Hashtable Collection class in C#.

```
using System;
using System.Collections;
namespace HasntableExample
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating Hashtable collection with default constructor
            Hashtable hashtable = new Hashtable();
            //Adding elements to the Hash table using key value pair
            hashtable.Add("EId", 1001); //Here key is Eid and value is 1001
            hashtable.Add("Name", "James"); //Here key is Name and value is J
            hashtable.Add("Job", "Developer");
            hashtable.Add("Salary", 3500);
            hashtable.Add("Location", "Mumbai");
            hashtable.Add("Dept", "IT");
            hashtable.Add("EmailID", "a@a.com");
            //Checking the key using the Contains method
            Console.WriteLine("Is EmailID Key Exists : " +
hashtable.Contains("EmailID"));
            Console.WriteLine("Is Department Key Exists : " +
hashtable.Contains("Department"));
            //Checking the key using the ContainsKey method
            Console.WriteLine("Is EmailID Key Exists : " +
hashtable.ContainsKey("EmailID"));
            Console.WriteLine("Is Department Key Exists : " +
hashtable.ContainsKey("Department"));
            //Checking the value using the ContainsValue method
            Console.WriteLine("Is Mumbai value Exists : " +
hashtable.ContainsValue("Mumbai"));
            Console.WriteLine("Is Technology value Exists : " +
hashtable.ContainsValue("Technology"));
            Console.ReadKey();
        }
    }
}
```

Output:

```
Is EmailID Key Exists : True
Is Department Key Exists : False
Is EmailID Key Exists : True
Is Department Key Exists : False
Is Mumbai value Exists : True
Is Technology value Exists : False
```

How to Remove Elements from a Non-Generic Hashtable Collection in C#?

If you want to remove an element from the Hashtable, then you can use the following Remove method of the C# Hashtable class.

1. **Remove(object key):** This method is used to remove the element with the specified key from the Hashtable. Here, the parameter key specifies the element to remove. It throws the KeyNotFoundException if the specified key is not found in the Hashtable, so check for an existing key using the ContainsKey() method before removing it.

If you want to remove all the elements from the hashtable, then you can use the following Clear method of the Hashtable class in C#.

2. **Clear():** This method is used to remove all elements from the Hashtable

For a better understanding of how to use the Remove and Clear method of the Non-Generic Hashtable collection class, please have a look at the below example.

```
using System;
using System.Collections;
namespace HasntableExample
{
    class Program
    {
        static void Main(string[] args)
        {
            Hashtable employee = new Hashtable
            {
                { "EId", 1001 },
                { "Name", "James" },
                { "Salary", 3500 },
                { "Location", "Mumbai" },
                { "EmailID", "a@a.com" }
            };
            //Count Property returns the number of elements present in the collection
            Console.WriteLine($"Hashtable Total Elements: {employee.Count}");
            // Remove EId as this key exists
            employee.Remove("EId");
            Console.WriteLine($"After Removing EID Total Elements: {employee.Count}");
            //Following statement throws run-time exception: KeyNotFoundException
            //employee.Remove("City");
            //Check key before removing it
            if (employee.ContainsKey("City"))
            {
                employee.Remove("City");
            }
            else
            {
                Console.WriteLine("Hashtable doesnot contain City key");
            }
        }
    }
}
```

```

        //Removes all elements
        employee.Clear();
        Console.WriteLine($"After Clearing Hashtable Total Elements:
{employee.Count}");
        Console.ReadKey();
    }
}
}

```

Output:

```

Hashtable Total Elements: 5
After Removing EID Total Elements: 4
Hashtable doesnot contain City key
After Clearing Hashtable Total Elements: 0

```

How to Assign Values to a Hashtable with Indexer in C#?

In order to add value to a Hashtable with an indexer, we need to use square brackets after the Hashtable name. This is because a Hashtable works with key/value pairs, we must have to specify both key and value while adding the elements. The key is specified between square brackets. The syntax is given below.

Syntax: `hashtable[key] = value;`

For a better understanding, please have a look at the below example. In the below example, we have added new values to an empty Hashtable with the indexer. Here, 1, 5, and 30 are the keys, and the One, Five, and Thirty are the values that correspond to each key respectively.

```

using System;
using System.Collections;
namespace HasntableExample
{
    class Program
    {
        static void Main(string[] args)
        {
            Hashtable hashtable = new Hashtable();
            hashtable[1] = "One";
            hashtable[5] = "Five";
            hashtable[30] = "Thirty";
            foreach (DictionaryEntry de in hashtable)
            {
                Console.WriteLine($"Key: {de.Key}, Value: {de.Value}");
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```
Key: 5, Value: Five
Key: 30, Value: Thirty
Key: 1, Value: One
```

How to Update a Hashtable in C# using Indexer?

We already discussed that we can retrieve the value from the Hashtable by using the key in the indexer. But the point that you need to remember is, that the Hashtable in C# is a non-generic collection class, so it is our responsibility to type-cast values to the appropriate type while retrieving it. In the same way, you can also use the key indexer to update an existing key value in Hashtable. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections;
namespace HashtableExample
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating Hashtable collection using collection-initializer syntax
            Hashtable employee = new Hashtable
            {
                { "EId", 1001 },
                { "Name", "James" },
                { "Salary", 3500 },
                { "Location", "Mumbai" },
                { "EmailID", "a@a.com" }
            };
            string EmployeeName = (string)employee["Name"]; //Cast to String
            int EmployeeSalary = (int)employee["Salary"]; //Cast to Int
            Console.WriteLine("Before Updating Name and Salary");
            Console.WriteLine($"Employee Name: {EmployeeName}");
            Console.WriteLine($"Employee Salary: {EmployeeSalary}");
            //Updating the Name and Salary
            employee["Name"] = "Smith"; //Update value of Name key
            employee["Salary"] = 5000; //Update value of Salary key
            Console.WriteLine("\nAfter Updating Name and Salary");
            EmployeeName = (string)employee["Name"];
            EmployeeSalary = (int)employee["Salary"];
            Console.WriteLine($"Employee Name: {EmployeeName}");
            Console.WriteLine($"Employee Salary: {EmployeeSalary}");

            Console.ReadKey();
        }
    }
}
```

Output:

```
Before Updating Name and Salary
Employee Name: James
Employee Salary: 3500

After Updating Name and Salary
Employee Name: Smith
Employee Salary: 5000
```

How to Clone a Non-Generic Hashtable Collection in C#?

If you want to clone the Non-Generic Hashtable collection in C#, then you need to use the following Clone() method which is provided by the Non-Generic Hashtable Collection Class.

1. **Clone():** This method is used to create and return a shallow copy of a Hashtable object.

For a better understanding, please have a look at the below example.

```
using System;
using System.Collections;
namespace HashtableExample
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating Hashtable collection with default constructor
            Hashtable hashtable = new Hashtable();
            //Adding elements to the Hash table using Add method
            hashtable.Add("EId", 1001);
            hashtable.Add("Name", "James");
            hashtable.Add("Job", "Developer");
            hashtable.Add("Salary", 3500);
            hashtable.Add("Location", "Mumbai");
            hashtable.Add("Dept", "IT");
            hashtable.Add("EmailID", "a@a.com");
            Console.WriteLine("Hashtable Elements:");
            foreach (DictionaryEntry item in hashtable)
            {
                Console.WriteLine($"Key: {item.Key}, Value: {item.Value}");
            }
            //Creating a clone Hashtable using Clone method
            Hashtable cloneHashtable = (Hashtable)hashtable.Clone();
            Console.WriteLine("\nCloned Hashtable Elements:");
            foreach (DictionaryEntry item in cloneHashtable)
            {
                Console.WriteLine($"Key: {item.Key}, Value: {item.Value}");
            }
        }
    }
}
```

```

        Console.ReadKey();
    }
}

```

Output:

```

Hashtable Elements:
Key: Salary, Value: 3500
Key: Location, Value: Mumbai
Key: EmailID, Value: a@a.com
Key: Dept, Value: IT
Key: EId, Value: 1001
Key: Job, Value: Developer
Key: Name, Value: James

Cloned Hashtable Elements:
Key: EId, Value: 1001
Key: Job, Value: Developer
Key: Location, Value: Mumbai
Key: Dept, Value: IT
Key: Name, Value: James
Key: Salary, Value: 3500
Key: EmailID, Value: a@a.com

```

How to Copy a Hashtable to an Existing Array in C#?

In order to copy a hashtable to an existing array in C#, we need to use the following CopyTo method of the Non-Generic Hashtable Collection Class.

1. **CopyTo(Array array, int arrayIndex):** The CopyTo method of the Non-Generic Hashtable Collection Class is used to copy hashtable elements to a one-dimensional Array object, starting at the specified index in the array. Here, the parameter array specifies the one-dimensional Array object that is the destination of the DictionaryEntry objects copied from the hashtable. The Array must have zero-based indexing. The arrayIndex parameter specifies the zero-based index in the array at which copying begins. If the parameter array is null, then it will throw ArgumentNullException. If the parameter arrayIndex is less than zero, then it will throw ArgumentOutOfRangeException.

The key/value pairs are copied to the Array object in the same order in which the enumerator iterates through the Hashtable object. This method is an O(n) operation, where n is Count.

1. To copy only the keys in the Hashtable, use Hashtable.Keys.CopyTo.
2. To copy only the values in the Hashtable, use Hashtable.Values.CopyTo.

For a better understanding, please have a look at the below example.

```

using System;
using System.Collections;
namespace HasntableExample
{
    class Program

```

```

{
    static void Main(string[] args)
    {
        //Creating Hashtable collection with default constructor
        Hashtable hashtable = new Hashtable();
        //Adding elements to the Hash table using Add method
        hashtable.Add("EId", 1001);
        hashtable.Add("Name", "James");
        hashtable.Add("Location", "Mumbai");
        //Printing All Queue Elements using For Each Loop
        Console.WriteLine("Hashtable Elements:");
        foreach (DictionaryEntry item in hashtable)
        {
            Console.WriteLine($"Key: {item.Key}, Value: {item.Value}");
        }
        //Copying the Hashtable to an object array
        DictionaryEntry[] myArray = new DictionaryEntry[hashtable.Count];
        hashtable.CopyTo(myArray, 0);
        Console.WriteLine("\nHashtable Copy Array Elements:");
        foreach (DictionaryEntry item in myArray)
        {
            Console.WriteLine($"Key: {item.Key}, Value: {item.Value}");
        }
        Object[] myObjArrayKey = new Object[hashtable.Count];
        Object[] myObjArrayValue = new Object[hashtable.Count];
        Console.WriteLine("\nCopyTo Method to Copy Keys:");
        hashtable.Keys.CopyTo(myObjArrayKey, 0);
        foreach (var key in myObjArrayKey)
        {
            Console.WriteLine($"{key} ");
        }
        Console.WriteLine("\nCopyTo Method to Copy Values:");
        hashtable.Values.CopyTo(myObjArrayValue, 0);
        foreach (var key in myObjArrayValue)
        {
            Console.WriteLine($"{key} ");
        }
        Console.ReadKey();
    }
}
}

```

Output:

```
Hashtable Elements:
Key: Name, Value: James
Key: EId, Value: 1001
Key: Location, Value: Mumbai

Hashtable Copy Array Elements:
Key: Name, Value: James
Key: EId, Value: 1001
Key: Location, Value: Mumbai

CopyTo Method to Copy Keys:
Name
EId
Location

CopyTo Method to Copy Values:
James
1001
Mumbai
```

Non-Generic Hashtable Collection Class Properties in C#

1. **IsFixedSize**: Gets a value indicating whether the System.Collections.Hashtable has a fixed size. It returns true if the Hashtable object has a fixed size; otherwise, false. The default is false.
2. **Keys**: Gets a System.Collections.ICollection containing the keys in the Hashtable. It returns a System.Collections.ICollection containing the keys in the Hashtable.
3. **IsSynchronized**: Gets a value indicating whether access to the Hashtable is synchronized (thread-safe). It returns true if access to the Hashtable is synchronized (thread-safe); otherwise, false. The default is false.
4. **IsReadOnly**: Gets a value indicating whether the Hashtable is read-only. It returns true if the Hashtable object is read-only; otherwise, false. The default is false.
5. **Count**: Gets the number of key/value pairs contained in the Hashtable. It returns the number of key/value pairs contained in the System.Collections.Hashtable.
6. **Values**: Gets a System.Collections.ICollection containing the values in the Hashtable. It returns a System.Collections.ICollection containing the values in the Hashtable.
7. **SyncRoot**: Gets an object that can be used to synchronize access to the Hashtable. It returns an object that can be used to synchronize access to the Hashtable.
8. **comparer**: Gets or sets the System.Collections.IComparer to use for the Hashtable. It returns the System.Collections.IComparer to use for the Hashtable.
9. **hcp**: Gets or sets the object that can dispense hash codes. It returns the object that can dispense hash codes.
10. **EqualityComparer**: Gets the System.Collections.IEqualityComparer to use for the Hashtable. It returns the System.Collections.IEqualityComparer to use for the Hashtable.

In the next article, I am going to discuss the [Non-Generic Stack Collection Class in C#](#) with Examples. Here, in this article, I try to explain the **Non-Generic Hashtable Collection Class in C#** with Examples. I hope this Non-Generic Hashtable Collection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.